MAS Final Project: Symmetry Reduction and Verification

1. Scope

This chapter documents the core verification pipeline centered on:

- `symmetry_reduction.py`
- `verification.py`

and how they integrate with:

- `main.py` (`--detect-symmetry`, `--verify-refine`)
- `refinement.py` (counterexample-driven constraint updates)
- `pathfinding.py` (planner under verification)

The goal is to explain exactly what is reduced, what is checked, and what conclusions are valid from the current implementation.

2. Why Symmetry Reduction Is Used

In multi-agent systems, many states are equivalent under relabeling of same-role agents.

Example:

- Agent A and Agent B are both idle.
- Swapping their identities without changing physical configuration should not create a meaningfully new verification state.

Without symmetry reduction, verification tracks both labeled variants separately, increasing state-space size.

With symmetry reduction, both map to one canonical quotient key.

3. Module Overview: `symmetry_reduction.py`

Key type aliases:

- `AgentKey = Tuple[int, int, int, int]`
- `OrbitKey = Tuple[AgentKey, ...]`
- `StateKey = Tuple[OrbitKey, ...]`

Core functions:

1. `_agent_role(robot) -> Tuple[int, int]`
2. `detect_role_orbits(robots) -> List[List[int]]`
3. `canonicalize_agents(robots, orbits) -> StateKey`
4. `canonicalize_state(env, include_shelves=False) -> (StateKey, shelf_key)`
5. `build_quotient_model(env) -> Dict[str, object]`

4. Role Abstraction

4.1 Role Encoding

`_agent_role(robot)` returns a compact role tuple:

- `(0, 0)` if robot carries nothing
- `(1, 1)` if robot carries a requested shelf
- `(1, 0)` if robot carries a non-requested shelf

Interpretation:

- first bit: carrying anything or not
- second bit: carrying requested item or not

This means symmetry is role-based, not identity-based.

4.2 Orbit Construction

`detect_role_orbits(robots)` groups robot indices by role.

Each orbit is a list of indices into the robot list.
All robots in the same orbit are considered permutation-equivalent for quotienting.

Important detail:

- Orbit grouping depends on current carry state.
- Orbit partition can change over time as robots pick/drop/deliver.

5. Canonicalization Logic

5.1 Agent Canonicalization

`canonicalize_agents(robots, orbits)`:

1. Converts each orbit member to:
   - `(x, y, dir_value, carrying_requested_flag)`
2. Sorts those tuples inside each orbit.
3. Returns tuple of sorted orbit tuples.

Effect:

- Any permutation within an orbit yields the same orbit key.

5.2 State Canonicalization

`canonicalize_state(env, include_shelves=False)`:

1. Detects orbits from current env robots.
2. Builds canonical agent key.
3. Optionally includes a canonical shelf key.

If `include_shelves=False`:

- returns `(agent_key, ())`

If `include_shelves=True`:

- scans uncarried shelves only,
- emits sorted tuples:
  - `(shelf_x, shelf_y, requested_flag)`

and returns `(agent_key, shelf_key)`.

Design implication:

- carried shelves are encoded through robot carry role, not shelf list.

6. Quotient Metadata Helper

`build_quotient_model(env)` returns:

- `orbits`: role-based groups of robot indices
- `representatives`: first index from each orbit
- `mapping`: robot index -> orbit index

This helper is used by CLI symmetry inspection (`main.py --detect-symmetry`).

7. What Is Reduced and What Is Not

7.1 Reduced

- Relabelings of agents inside same role orbit.

7.2 Not Reduced

- Differences across roles (idle vs carrying requested are distinct).
- Geometric/map automorphisms (rotations/reflections are not modeled).
- Direction differences (`dir` stays in key).
- Requested-status shelf differences (when shelves included).

So this is role-permutation symmetry reduction, not full structural symmetry of the map.

8. Verification Module: `verification.py`

Primary function:

- `verify_on_quotient(env, planner, horizon, trials, include_shelves, min_separation, progress_every, logger)`

Helper:

- `_min_pairwise_manhattan(robots)`

9. Verification Algorithm

For each trial:

1. `env.reset()`
2. Initialize:
   - `visited` quotient keys,
   - `trace` (robot positions over time),
   - `action_history`
3. Check initial separation against `min_separation`.
4. For each step up to `horizon`:
   - compute `state_key = canonicalize_state(...)`
   - if repeated key in `visited`, stop this trial (cycle reached)
   - else add key and execute one planner/environment step
   - collect collisions/conflicts and update safety margin
   - early return unsafe on:
     - collisions / explicit conflicts
     - separation violation

If all trials finish without unsafe event:

- returns safe summary with:
  - `safe=True`
  - `delta_q`
  - `avg_collision_rate`

10. Safety Metrics and Outputs

10.1 `delta_q`

`delta_q = min_observed_pairwise_distance - min_separation`

Interpretation:

- `delta_q > 0`: safety margin above threshold
- `delta_q = 0`: exactly at threshold
- `delta_q < 0`: threshold violated

10.2 `avg_collision_rate`

`total_collisions / total_steps` across verification rollouts.

10.3 Unsafe Return Payload

When unsafe, returns:

- `safe=False`
- `counterexample` (position trace)
- `actions` (action list per step)
- `conflicts` (from env conflict records or synthetic separation record)

- `delta_q`
- `avg_collision_rate` (except initial-time immediate separation return path)

11. Interaction with Environment Conflict Semantics

Verification consumes `env.last_conflicts`, which is generated by environment intent analysis.

Conflict types observed:

- `boundary`
- `vertex`
- `edge`

These are generated from intended forward moves before final conflict resolution in `env.step(...)`.

This gives verifier/refiner structured conflict signals, not just scalar collision counts.

12. Counterexample-Guided Refinement Link

When `verify_on_quotient` returns unsafe:

- `main.py` passes `conflicts` and `counterexample` to `refine_planner_with_conflicts(...)`.

Refinement then adds constraints to planner:

- vertex conflict -> forbidden position at `t+1`
- edge conflict -> forbidden in both directions at `t+1`
- boundary conflict -> forbidden destination at `t+1`

Next iteration reruns verification with modified planner constraints.

13. CLI Entry Points

13.1 Symmetry Inspection

```
python main.py --detect-symmetry
```

Prints orbit groups and quotient metadata from a fresh reset state.

13.2 Verification + Refinement

```
python main.py --verify-refine --verify-horizon 30 --verify-trials 20
```

Optional shelf-aware quotient:

```
python main.py --verify-refine --verify-include-shelves
```

14. Correctness Properties You Can Assert

Given current code, these statements are accurate:

1. Canonicalization is invariant to permutations of same-role agents.
2. States with role differences are not merged by quotienting.
3. Verification uses quotient keys for visited-state detection at every step.
4. Unsafe runs return actionable traces/conflict payloads for refinement.

15. What Verification Does Not Prove

Current verifier is bounded and simulation-based, so it does not prove:

- global unbounded safety,
- safety under all possible stochastic initializations beyond tested trials,
- liveness/fairness guarantees beyond empirical behavior,
- completeness of conflict vocabulary for all hazardous scenarios.

16. Practical Guidance for Experiments

For reproducible comparisons:

1. Fix seed via `--seed`.
2. Keep planner/config fixed.
3. Run with and without `--verify-include-shelves`.
4. Log:
   - safe/unsafe outcome,
   - `delta_q`,
   - `avg_collision_rate`,
   - number of applied constraints per iteration.

For symmetry-specific impact:

- Compare number of visited keys under:
  - labeled raw state tracking (custom instrumentation),
  - quotient key tracking (`canonicalize_state`).

17. Known Design Tradeoffs

1. Role abstraction is intentionally simple and fast.
2. It may under-capture symmetry in some domains and over-partition in others.
3. Including shelves in quotient key increases fidelity but can reduce compression.
4. Excluding shelves improves compression but can merge states that differ in shelf layout relevance.

18. Risks and Edge Cases

1. Orbit list ordering follows first-seen role order; canonicalization remains deterministic for fixed robot ordering.
2. If environment robot ordering changes unexpectedly between equivalent states, quotient behavior may differ unless upstream ordering is controlled.
3. Initial-separation failure path returns early before average-rate accumulation.

19. Suggested Next Enhancements

1. Add dedicated unit tests for symmetry functions:
   - permutation invariance,
   - role separation,
   - shelf-key invariance.
2. Add benchmark harness for quotient compression ratio.
3. Add optional stronger role signature (task assignment class, proximity class, etc.) for finer equivalence control.
4. Add formal docs for expected conflict schema consumed by refinement.

20. Chapter Summary

Symmetry reduction in this project is a role-orbit canonicalization layer that feeds bounded verification through quotient-state cycle detection.
Verification returns structured unsafe evidence, which is converted into hard planner constraints through refinement.
Together, this creates a practical engineering loop where symmetry-aware abstraction reduces redundant verification states while still driving actionable safety improvements.